

### **Program-3 : Code a Dockerized Python Flask Application**

#### **Description :**

This program creates a simple Flask web application and deploys it inside a Docker container to ensure portability and environment consistency.

#### **Step-1 : Create Program Directory**

```
mkdir Program-3
```

```
cd Program-3
```

#### **Step-2 : Create Dockerfile**

```
nano Dockerfile
```

```
FROM python:latest
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

#### **Step-3 : Create app.py**

```
nano app.py
```

```
from flask import Flask
```

```
app = Flask(__name__)
```

```
@app.route("/")
```

```
def home():
```

```
    return "Hello from Simple Flask Docker!"
```

```
if __name__ == "__main__":
```

```
    app.run(host="0.0.0.0", port=5000)
```

#### **Step-4 : Create requirements.txt**

```
nano requirements.txt
```

```
Flask==2.3.3
```

#### **Step-5 : Build Docker Image**

```
sudo docker build -t program-3 .
```

#### **Step-6 : Run Docker Container**

sudo docker run -p 5000:5000 program-3

```
pavan-k@pavank1RV24MC072:~$ mkdir Program-3
cd Program-3
pavan-k@pavank1RV24MC072:~/Program-3$ nano Dockerfile
pavan-k@pavank1RV24MC072:~/Program-3$ nano app.py
pavan-k@pavank1RV24MC072:~/Program-3$ nano requirements.txt
pavan-k@pavank1RV24MC072:~/Program-3$ sudo docker build -t program-3 .
[+] Building 50.8s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 192B
=> [internal] load metadata for docker.io/library/python:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:latest@sha256:6d58c1a9444bc2664f0fa20c43a592fcd62698eb9a9c32257516538a2746c19a
=> => resolve docker.io/library/python:latest@sha256:6d58c1a9444bc2664f0fa20c43a592fcd62698eb9a9c32257516538a2746c19a
=> sha256:1227bf08bd428f8324f592cf370b34de4c11dd57aba693e8f3ec9ccbdad8d468d 249B / 249B
=> sha256:ad1077facb26b82208d59d5550cbbab6af9cf38c971f00a43e3d3cda71ffe76a 29.40MB / 29.40MB
=> sha256:b1741a62cceed4e959c504d7a6a25e533c5783994c386b24d7b31a646163283d 6.00MB / 6.00MB
=> sha256:02e37abc533a09f86edcf3602251a61a6208ddaa131e1a1d5757e5f94a54d645 235.98MB / 235.98MB
=> sha256:378c64c4458002be86f2d86c5768ae9ec0cf69afac7d1444e50206dc4566fa9 67.78MB / 67.78MB
=> sha256:15f14138abe4f09d3ef3953105144728056046ae469ae21e8e42749bacd67cca 25.61MB / 25.61MB
=> sha256:281b80c799ded5b9a390d2e8c59938db01ee395ab809dd34259897c660751f4e 49.29MB / 49.29MB
=> extracting sha256:281b80c799ded5b9a390d2e8c59938db01ee395ab809dd34259897c660751f4e
=> extracting sha256:15f14138abe4f09d3ef3953105144728056046ae469ae21e8e42749bacd67cca
=> extracting sha256:378c64c4458002be86f2d86c5768ae9ec0cf69afac7d1444e50206dc4566fa9
=> extracting sha256:02e37abc533a09f86edcf3602251a61a6208ddaa131e1a1d5757e5f94a54d645
=> extracting sha256:b1741a62cceed4e959c504d7a6a25e533c5783994c386b24d7b31a646163283d
=> extracting sha256:ad1077facb26b82208d59d5550cbbab6af9cf38c971f00a43e3d3cda71ffe76a
=> extracting sha256:1227bf08bd428f8324f592cf370b34de4c11dd57aba693e8f3ec9ccbdad8d468d
=> [internal] load build context
=> => transferring context: 475B
=> [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt .
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY . .
=> exporting to image
=> => exporting layers
=> exporting manifest sha256:1f8c0b1c1b0b62a4e6d39ccbcff51e32f0ed2f63438d4bc39debdac1d0a00bf
=> exporting config sha256:ab64feafa9453b26b9bbdddc21325c6b925df646998435d6f6ebba39d1e44574
=> exporting attestation manifest sha256:13fe4d16f5356fcd957f1b66c258586da4df9b7799266561859597869c42a68
=> exporting manifest list sha256:c4483e9a3ad41671dd32d5410041b23f6efcd833ebfb8a141bb6c5e3feb2a0e6
=> naming to docker.io/library/program-3:latest
=> => unpacking to docker.io/library/program-3:latest
pavan-k@pavank1RV24MC072:~/Program-3$ sudo docker run -p 5000:5000 program-3
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
```

## Step-7 : Verify Output

http://localhost:5000

Hello from Simple Flask Docker!



## Step-8 : Stop and Remove Container and Image

sudo docker container ls -a

sudo docker container stop <container-id>

sudo docker container rm <container-id>

sudo docker image rm program-3

## Result :

The Flask application is successfully containerized using Docker and executed through port mapping.